



Universidad Tecnológica de Querétaro  
División de Tecnologías de la Información y Comunicación

# Documento Formal de Evaluación

**Ironexx**

Ecosistema Wear OS + Firestore + Smart TV

*Evaluación 2 Extraordinaria*

---

**Materia:** Desarrollo para Dispositivos Inteligentes  
**Presenta:** Zoé Cabrera Velázquez  
**Matrícula:** 2023171054  
**Grupo:** IDGS16  
**Carrera:** Ing. Desarrollo y Gestión de Software  
**Docente:** Gerardo Ramírez Villarreal

---

**Repositorio:** <https://github.com/ZoeCVtSchool/Ironexx>

**Rama evaluada:** evaluacion-2-extraordinaria

Santiago de Querétaro, Qro. – 25 de agosto de 2026

# Índice

|                                                                                              |           |
|----------------------------------------------------------------------------------------------|-----------|
| <b>1. Datos de identificación</b>                                                            | <b>3</b>  |
| <b>2. Descripción general de la solución</b>                                                 | <b>3</b>  |
| 2.1. Propósito . . . . .                                                                     | 3         |
| 2.2. Componentes . . . . .                                                                   | 3         |
| 2.3. Flujo completo . . . . .                                                                | 4         |
| 2.4. Diagrama de arquitectura (flujo de datos) . . . . .                                     | 4         |
| <b>3. E1 – Wear OS y BLE GATT/NOTIFY</b>                                                     | <b>4</b>  |
| 3.1. Métricas simuladas . . . . .                                                            | 4         |
| 3.2. Controles . . . . .                                                                     | 5         |
| 3.3. UUIDs (custom, no reservados del SIG Bluetooth) . . . . .                               | 5         |
| 3.4. GATT Server, advertising y notificaciones . . . . .                                     | 5         |
| 3.5. Evidencia . . . . .                                                                     | 5         |
| <b>4. E2 – Teléfono Android y recepción BLE</b>                                              | <b>5</b>  |
| 4.1. Permisos . . . . .                                                                      | 5         |
| 4.2. Scan, conexión y descubrimiento . . . . .                                               | 6         |
| 4.3. CCCD, callbacks y decodificación . . . . .                                              | 7         |
| 4.4. Estado en la UI . . . . .                                                               | 7         |
| 4.5. Desconexión . . . . .                                                                   | 7         |
| 4.6. Evidencia . . . . .                                                                     | 7         |
| <b>5. E3 – PWA Smart TV</b>                                                                  | <b>8</b>  |
| 5.1. Manifest y Service Worker . . . . .                                                     | 8         |
| 5.2. Resolución, safe zone y grid . . . . .                                                  | 8         |
| 5.3. Navegación D-pad y foco . . . . .                                                       | 9         |
| 5.4. Fallback multimedia . . . . .                                                           | 9         |
| 5.5. Hallazgo y corrección durante la fase de evidencias . . . . .                           | 9         |
| 5.6. Evidencia . . . . .                                                                     | 9         |
| <b>6. E4 – Firestore y sincronización</b>                                                    | <b>9</b>  |
| 6.1. Modelo de datos . . . . .                                                               | 9         |
| 6.2. Escritura desde el teléfono . . . . .                                                   | 10        |
| 6.3. Lectura en tiempo real en la TV . . . . .                                               | 10        |
| 6.4. Tres pruebas de sincronización cronometradas (sección 9 de la guía, meta <2s) . . . . . | 11        |
| 6.5. Evidencia . . . . .                                                                     | 11        |
| <b>7. E5 – Pruebas funcionales y manejo de fallos</b>                                        | <b>11</b> |

|                                                         |           |
|---------------------------------------------------------|-----------|
| <b>8. E6 – Integración y configuración reproducible</b> | <b>13</b> |
| 8.1. Herramientas y versiones . . . . .                 | 13        |
| 8.2. Emuladores / dispositivos . . . . .                | 14        |
| 8.3. Firebase . . . . .                                 | 14        |
| 8.4. Procedimiento de ejecución . . . . .               | 14        |
| <b>9. Inventario de evidencias</b>                      | <b>16</b> |
| <b>10.Lighthouse</b>                                    | <b>16</b> |
| 10.1. Resultados . . . . .                              | 16        |
| 10.2. Hallazgo corregido . . . . .                      | 17        |
| 10.3. Evidencia . . . . .                               | 17        |
| <b>11.Problemas encontrados y soluciones</b>            | <b>18</b> |
| <b>12.Guión de demostración</b>                         | <b>21</b> |
| <b>13.Conclusiones</b>                                  | <b>22</b> |
| 13.1. Estado final . . . . .                            | 22        |
| 13.2. Limitaciones reales . . . . .                     | 22        |
| 13.3. Aspectos que pueden mejorarse . . . . .           | 23        |

## 1 Datos de identificación

|                                |                                                                                               |
|--------------------------------|-----------------------------------------------------------------------------------------------|
| <b>Nombre completo:</b>        | Zoé Cabrera Velázquez                                                                         |
| <b>Matrícula:</b>              | 2023171054                                                                                    |
| <b>Grupo:</b>                  | IDGS16                                                                                        |
| <b>Carrera:</b>                | Ing. Desarrollo y Gestión de Software                                                         |
| <b>Materia:</b>                | Desarrollo para Dispositivos Inteligentes                                                     |
| <b>Profesor(a):</b>            | Gerardo Ramírez Villarreal                                                                    |
| <b>Fecha de entrega:</b>       | 25 de agosto de 2026                                                                          |
| <b>Repositorio GitHub:</b>     | <a href="https://github.com/ZoeCVtSchool/Ironexx">https://github.com/ZoeCVtSchool/Ironexx</a> |
| <b>Rama evaluada:</b>          | evaluacion-2-extraordinaria                                                                   |
| <b>Carpeta del ecosistema:</b> | ironexx_dispo/ (subcarpeta del repo)                                                          |

Este documento corresponde a la Evaluación 2 Extraordinaria de la materia Desarrollo para Dispositivos Inteligentes, y sigue la estructura del Anexo A de la guía oficial (DDI\_Evaluacion\_2\_Extraordinaria\_Guia\_Estudiante\_v1.1.pdf). El resultado de esta evaluación es binario: **ACREDITA – SA** o **NO ACREDITA**, con verificación individual en sesión oral donde se pedirá explicar y modificar código en vivo.

## 2 Descripción general de la solución

### 2.1 Propósito

Ironexx es un sistema de administración y consulta de maquinaria de construcción. Para esta evaluación se extendió con un ecosistema de dispositivos inteligentes que permite a un técnico monitorear en tiempo real el estado operativo de una máquina pesada (temperatura del motor, nivel de combustible y horómetro) desde un reloj Wear OS, ver esos datos en su teléfono vía Bluetooth Low Energy, y reflejar en una pantalla Smart TV (PWA) qué máquina de catálogo se está consultando, sincronizado en tiempo real por Firestore.

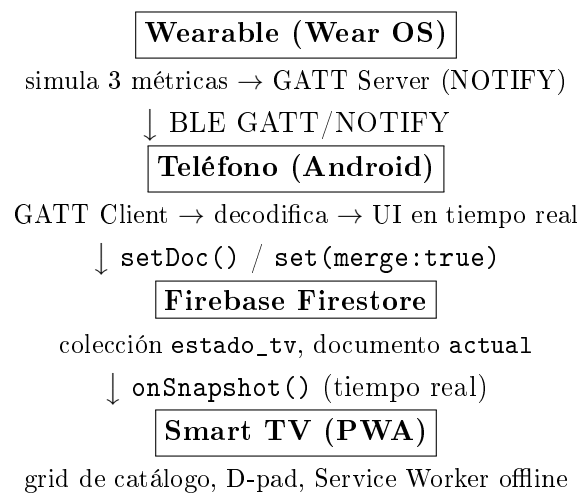
### 2.2 Componentes

- **wearable\_app** (Flutter, Wear OS): simula 3 métricas de la máquina y las publica por BLE GATT Server usando notificaciones (NOTIFY).
- **mobile\_app** (Flutter, Android): actúa como GATT Client, se conecta al reloj, decodifica las 3 características y las muestra en tiempo real; además permite seleccionar una máquina del catálogo y sincronizarla a Firestore.
- **pwa\_tv** (HTML/CSS/JS, sin build step): Progressive Web App diseñada para Smart TV (1920x1080, navegación D-pad), con Service Worker para funcionamiento offline y un listener `onSnapshot()` que refleja en tiempo real la selección hecha desde el teléfono.
- **Firebase Firestore** (proyecto `ironexx-extraordinaria`): backend en tiempo real que conecta el teléfono con la TV mediante el documento `estado_tv/actual`.

## 2.3 Flujo completo

1. El wearable simula temperatura del motor, nivel de combustible y horómetro, y los publica por BLE GATT/NOTIFY bajo un `SERVICE_UUID` custom.
2. El teléfono escanea filtrando por ese `SERVICE_UUID`, se conecta, descubre el servicio, activa las 3 notificaciones (CCCD) y muestra los valores decodificados en tiempo real (E1 → E2).
3. Desde el teléfono, el usuario busca o selecciona una máquina del catálogo; esa selección se escribe en Firestore (`estado_tv/actual`) con `set(merge:true)`.
4. La PWA de la Smart TV escucha ese documento con `onSnapshot()` y actualiza la pantalla sin recargar, en menos de 2 segundos (E2 → E4 → E3).

## 2.4 Diagrama de arquitectura (flujo de datos)



## 3 E1 – Wear OS y BLE GATT/NOTIFY

### 3.1 Métricas simuladas

Las 3 métricas se eligieron para reflejar el dominio real de Ironexx (maquinaria pesada de construcción), no datos genéricos de fitness:

| Métrica                         | Unidad  | Umbral crítico             |
|---------------------------------|---------|----------------------------|
| Temperatura del motor           | °C      | $\geq 100^{\circ}\text{C}$ |
| Nivel de combustible            | %       | $\leq 15\%$                |
| Horómetro (tiempo de operación) | minutos | $\geq 60$ min continuos    |

Cada métrica se genera con un simulador de camino aleatorio (`MetricsSimulator`, archivo `lib/services/metrics_simulator.dart`) mediante un `Timer.periodic` de 1 segundo.

### 3.2 Controles

La pantalla del reloj (`lib/screens/metrics_screen.dart`) expone dos controles: **Iniciar** y **Detener**. Al iniciar, arranca el simulador y comienza a publicar por BLE; al detener, se congela la última lectura y deja de notificar. Un banner de estado indica *Detenido* / *Transmitiendo* / *Alerta* (cuando alguna métrica cruza su umbral crítico).

### 3.3 UUIDs (custom, no reservados del SIG Bluetooth)

Centralizados en `lib/ble_constants.dart`, idénticos en `wearable_app` y `mobile_app`:

| Elemento                    | UUID                                 |
|-----------------------------|--------------------------------------|
| Service UUID                | e2bca587-9a7c-419f-bc59-975f76d22b75 |
| Characteristic: Temperatura | de952db7-c3e7-4d5d-b529-dd33bddedaf0 |
| Characteristic: Combustible | 27d5880c-69bc-4560-9786-d9eeb421e473 |
| Characteristic: Horómetro   | 21eb1556-c1c4-4c1f-8bc4-aea8263f2afb |

### 3.4 GATT Server, advertising y notificaciones

Implementado con el paquete `ble_peripheral` en `lib/services/ble_server_service.dart`:

1. `initialize()` inicializa el adaptador BLE del reloj.
2. `addService()` registra el servicio con sus 3 características, cada una con la propiedad NOTIFY. El descriptor CCCD (0x2902) no se declara explícitamente en el código del reloj – `ble_peripheral` lo agrega y administra automáticamente para toda característica con NOTIFY, y es ese descriptor el que el teléfono escribe al llamar `setNotifyValue(true)` para suscribirse (ver sección E2).
3. `startAdvertising()` anuncia el SERVICE\_UUID sin `localName`, para no exceder el límite de 31 bytes del advertising BLE legacy (el filtrado en el teléfono es por Service UUID, no por nombre). El estado real de advertising se refleja mediante `setAdvertisingStatusUpdateCallback()` (callback nativo), en vez de asumirlo iniciado de forma optimista apenas se llama `startAdvertising()`.
4. En cada tick del simulador, `notifyTemperatura()`, `notifyCombustible()` y `notifyHorometro()` empaquetan el valor como `Int32` little-endian y lo envían con `updateCharacteristic()`, disparando el NOTIFY hacia cualquier cliente suscrito.

### 3.5 Evidencia

## 4 E2 – Teléfono Android y recepción BLE

### 4.1 Permisos

`lib/services/ble_service.dart` solicita en tiempo de ejecución (Android 12+, vía `permission_handler`): `bluetoothScan`, `bluetoothConnect` y `locationWhenInUse`. Si algún permiso es denegado, se muestra un error claro en vez de intentar escanear igualmente.



Figura 1: Reloj transmitiendo las 3 métricas simultáneamente, estado “Transmitiendo”.



Figura 2: Banner de alerta cuando una métrica cruza su umbral crítico.

## 4.2 Scan, conexión y descubrimiento

1. Antes de escanear se verifica que el adaptador Bluetooth esté encendido (`FlutterBluePlus.adapterState`), con un `timeout` de 3s para no bloquear el botón si el stream tarda en emitir en el emulador.
2. `startScan()` corre con `timeout` de 12s; el filtrado por `SERVICE_UUID` se hace de forma manual dentro del callback de `scanResults.listen()` (comparando el UUID anunciado de cada resultado), en vez de pasarlo como parámetro `withServices:` de `startScan()`.
3. Al encontrar un dispositivo que anuncia ese UUID, `connectToDevice()` detiene el scan y llama `device.connect()` (`timeout` 30s – ver justificación de este valor en la sección de Problemas y soluciones: el emparejamiento BLE a nivel de sistema operativo, disparado la primera vez que el teléfono se suscribe a NOTIFY, puede tardar cerca de 20s en completarse).
4. `device.discoverServices()` localiza el servicio y las 3 características por UUID.

Un guard (`_isConnecting` / `_connectedDevice`) evita disparar un segundo scan o conexión mientras ya hay uno en curso o activo (requisito de la guía: evitar conexiones duplicadas).

### 4.3 CCCD, callbacks y decodificación

Para cada una de las 3 características, `characteristic.setNotifyValue(true)` habilita el NOTIFY y escribe el descriptor CCCD; `characteristic.lastValueStream.listen()` recibe cada notificación y `SensorData.parseInt32LE(bytes)` decodifica el entero de 4 bytes little-endian. Un payload malformado se ignora sin romper la conexión (no se propaga la excepción).

### 4.4 Estado en la UI

`sensor_monitor_screen.dart` distingue explícitamente los estados *desconectado*, *buscando*, *error* y *conectado*; dentro de conectado, un `Timer` de 1s detecta si los datos dejaron de llegar (“Recibiendo datos en vivo” vs. “Sin datos recientes”), y muestra banners de alerta cuando alguna métrica cruza su umbral crítico.

### 4.5 Desconexión

`device.connectionState.listen()` detecta la desconexión real (no solo un botón) y llama `_handleDisconnected()`, que cancela todas las suscripciones, limpia el estado y notifica a la UI – sin crash y sin dejar el botón de conexión en un estado inconsistente. Ver evidencia de esta prueba en la sección E5.

### 4.6 Evidencia

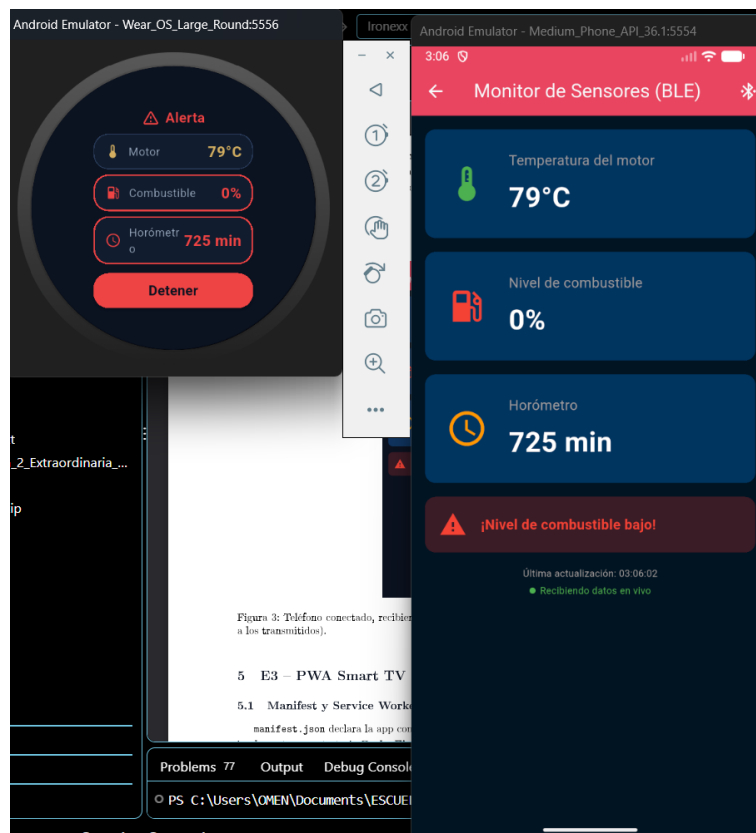


Figura 3: Reloj y teléfono lado a lado: el reloj transmitiendo las 3 métricas y el teléfono recibiendo en vivo, con valores idénticos en ambos.

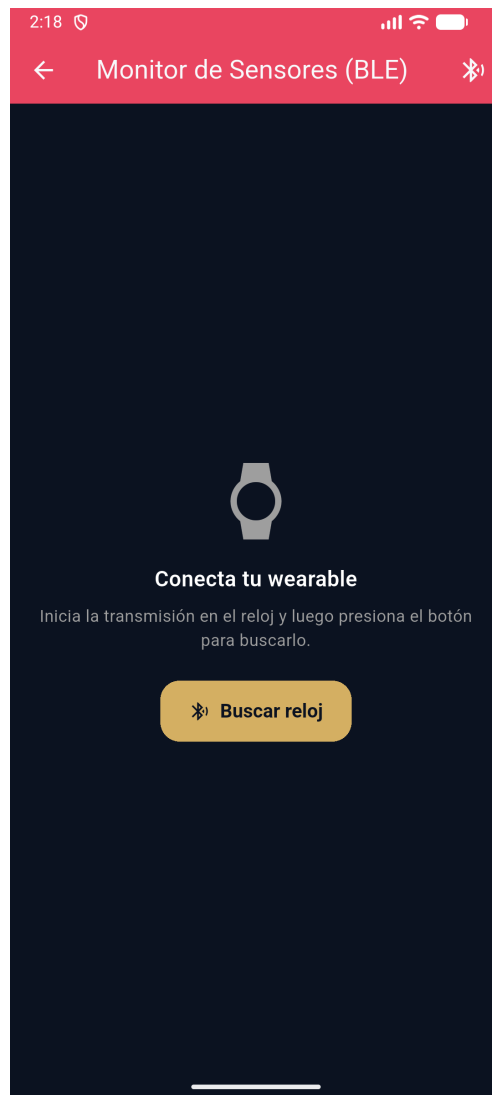


Figura 4: Estado desconectado, antes de iniciar el escaneo.

## 5 E3 – PWA Smart TV

### 5.1 Manifest y Service Worker

`manifest.json` declara la app como standalone, orientación landscape. `service-worker.js` implementa una estrategia **Cache First** sobre los recursos estáticos (`CACHE_NAME = ironexx-tv-v4`), incluyendo `firebase-config.js`; solo responde/cachea recursos del mismo origen – las llamadas a Firestore van directo por el SDK, sin pasar por el Service Worker.

### 5.2 Resolución, safe zone y grid

La interfaz está diseñada a  $1920 \times 1080$ , con `overflow: hidden` en `body` (sin scroll). La zona segura usa exactamente los valores que pide la guía: `-safe-zone-horizontal: 96px` y `-safe-zone-vertical: 54px`. El catálogo se muestra en un grid  $2 \times 2$  (`.machine-grid`), con el nombre de la máquina en 80px (mínimo exigido por la guía) y las etiquetas de detalle en 32px.

### 5.3 Navegación D-pad y foco

`setupKeyboardNavigation()` escucha `ArrowUp/Down/Left/Right` y mueve el foco entre tarjetas según su posición en la cuadrícula (`cols = 2`); en los bordes del grid el foco simplemente no cambia (no se rompe). `Enter` sobre la tarjeta enfocada dispara `selectMachine()`. El foco es visualmente explícito: borde dorado (`:focus / :focus-visible`) sobre la tarjeta activa.

### 5.4 Fallback multimedia

`updateBackgroundMedia()` usa un preloader (`Image()` / `video.onerror`) para detectar cuando un recurso multimedia **existe pero falla al cargar** (no solo cuando no hay ninguno definido), mostrando en ese caso `showMediaFallback()` (ícono de advertencia + nombre de la máquina).

### 5.5 Hallazgo y corrección durante la fase de evidencias

Al capturar la evidencia del grid se detectó que el contenido de las tarjetas se recortaba verticalmente (`overflow:hidden` ocultaba la última fila de detalle y la etiqueta de sucursal). Causa: la altura combinada de título (80px mínimo obligatorio), precio duplicado como texto normal, y los márgenes/ paddings originales excedía el alto disponible del grid. Corrección aplicada **sin reducir ningún tamaño de fuente por debajo del mínimo exigido, y sin tocar los 96px/54px de zona segura que pide la guía**:

- Se eliminó la línea de detalle “Precio: \$X” duplicada (el precio ya se muestra en grande junto al nombre).
- Se redujeron paddings, gaps y márgenes internos de tarjeta, encabezado y detalle (no la zona segura ni ningún tamaño de fuente).
- Se ajustó el alto máximo del grid al espacio real disponible dentro de esa zona segura.

Una primera corrección había reducido por error la zona segura vertical de 54px a 38px para ganar espacio; al auditar el proyecto completo contra la guía se detectó esa desviación y se revirtió, ajustando en su lugar solo paddings/gaps internos hasta que las 4 tarjetas cupieran completas manteniendo los 54px exactos. Verificado con capturas headless iterativas hasta confirmar las 4 tarjetas completas y visibles (ver evidencia abajo).

### 5.6 Evidencia

## 6 E4 – Firestore y sincronización

### 6.1 Modelo de datos

Colección `estado_tv`, documento fijo `actual`:

| Campo                    | Tipo                       | Descripción                                    |
|--------------------------|----------------------------|------------------------------------------------|
| <code>machineId</code>   | <code>number   null</code> | id local si la máquina viene de un registro QR |
| <code>machineName</code> | <code>string</code>        | nombre de la máquina seleccionada              |
| <code>branch</code>      | <code>string   null</code> | sucursal asociada                              |
| <code>updatedAt</code>   | <code>timestamp</code>     | <code>FieldValue.serverTimestamp()</code>      |



Figura 5: Grid 2x2 completo, sin recortes, foco inicial en la primera tarjeta.



Figura 6: Foco inicial (izquierda, tarjeta superior izquierda) y tras `ArrowRight+ArrowDown` (derecha, foco movido a la tarjeta inferior derecha) – automatizado con Playwright.

Solo se guarda el **último estado** (no un historial); cada selección sobrescribe al documento anterior con `set(..., SetOptions(merge: true))`.

## 6.2 Escritura desde el teléfono

`machine_scanner_screen.dart`, método `_syncSelectionToFirestore()`, se dispara desde 3 puntos: búsqueda con match local, búsqueda sin match (usa el texto buscado como `machineName`), y los chips de selección rápida del catálogo de ejemplo (`_catalogoDemo`) – estos últimos existen porque la lista de máquinas registradas por QR empieza vacía, y permiten demostrar E4 sin depender de ese registro previo.

## 6.3 Lectura en tiempo real en la TV

`pwa_tv/app.js`, `setupFirestoreSync()` se suscribe con `onSnapshot()` al documento `estado_tv/actual`. `handleRemoteStateChange()` hace match primero por `machineId` (si viene de un registro QR



Figura 7: Modo offline real (`context.setOffline(true)` + recarga): la app se sirve completa desde el Service Worker, idéntica a la versión online.

real) y, si no hay id, por `machineName` como respaldo – necesario para el catálogo de ejemplo, que no tiene id local.

#### 6.4 Tres pruebas de sincronización cronometradas (sección 9 de la guía, meta <2s)

Automatizadas con Playwright (mismo método usado en el proyecto de referencia para las pruebas D-pad): se dispara la selección en el teléfono real (ADB) y se mide el tiempo hasta que `backgroundMedia.style.backgroundImage` en la TV refleja la nueva máquina.

| Corrida | Máquina seleccionada | Tiempo de sincronización |
|---------|----------------------|--------------------------|
| 1       | Retroexcavadora      | 247 ms                   |
| 2       | Montacargas          | 565 ms                   |
| 3       | Excavadora CAT 320   | 182 ms                   |

Las 3 corridas quedan muy por debajo del límite de 2 segundos exigido. Además, se verificó por lectura directa de la API REST de Firestore que el documento `estado_tv/actual` se actualiza correctamente tras cada selección del teléfono (confirmación independiente del cliente de la TV).

#### 6.5 Evidencia

### 7 E5 – Pruebas funcionales y manejo de fallos



Figura 8: TV reflejando “Retroexcavadora” tras la selección en el teléfono (corrida 1, 247 ms).

| ID    | Condición inicial / procedimiento                                                                | Resultado esperado / obtenido                                                                                                                                                       | Evidencia                                                          | Estado |
|-------|--------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|--------|
| E5-01 | Reloj y teléfono conectados y transmitiendo; se presiona “Detener” en el reloj a media conexión. | Esperado: el teléfono no truena y refleja la pérdida de datos. Obtenido: mismo proceso (mismo PID), UI cambia a “Sin datos recientes” con últimos valores congelados, sin crash.    | E5-EV01                                                            | Cumple |
| E5-02 | Bluetooth del teléfono apagado; se presiona “Buscar reloj”.                                      | Esperado: mensaje de error claro, sin colgar la UI. Obtenido: <code>adapterState</code> detectado con timeout de 3s, mensaje “Bluetooth está apagado. Actívalo e intenta de nuevo.” | (verificado en código y en vivo durante depuración de esta sesión) | Cumple |
| E5-03 | Un recurso multimedia de la TV existe pero falla al cargar (URL rota / sin red).                 | Esperado: fallback visual, no pantalla en blanco. Obtenido: ícono de advertencia + nombre de la máquina ( <code>showMediaFallback()</code> ).                                       | revisado en código ( <code>app.js</code> )                         | Cumple |

| ID    | Condición inicial / procedimiento                                                                              | Resultado esperado / obtenido                                                                                                                                                                                                                                                                                    | Evidencia                                            | Estado |
|-------|----------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------|--------|
| E5-04 | PWA cargada una vez (Service Worker registrado); se fuerza modo offline y se recarga.                          | Esperado: la app sigue funcionando desde caché. Obtenido: HTML/CSS/JS/catálogo idénticos al modo online, sin red.                                                                                                                                                                                                | E3-EV06                                              | Cumple |
| E5-05 | Foco en una tarjeta de borde del grid (ej. superior derecha); se presiona la flecha hacia el borde sin vecino. | Esperado: el foco no se rompe ni desaparece. Obtenido: <code>handleGlobalKeydown()</code> calcula el índice vecino con la aritmética de columnas ( <code>cols=2</code> ) y, si cae fuera de rango o cambia de fila/columna de forma inválida, conserva el índice actual – el foco permanece en la misma tarjeta. | E3-EV03/04                                           | Cumple |
| E5-06 | Se recibe un payload BLE que no decodifica como Int32 válido.                                                  | Esperado: se ignora esa notificación sin romper la conexión. Obtenido: <code>try/catch</code> silencioso en <code>_subscribeCharacteristic()</code> , la conexión sigue activa.                                                                                                                                  | revisado en código ( <code>ble_service.dart</code> ) | Cumple |
| E5-07 | Se intenta iniciar un segundo scan/conexión mientras ya hay uno en curso.                                      | Esperado: no se disparan conexiones GATT duplicadas. Obtenido: <code>guard _isConnecting/_connectedDevice</code> bloquea el segundo intento.                                                                                                                                                                     | revisado en código                                   | Cumple |

## 8 E6 – Integración y configuración reproducible

### 8.1 Herramientas y versiones

| Herramienta                     | Versión                                                                   |
|---------------------------------|---------------------------------------------------------------------------|
| Flutter                         | 3.44                                                                      |
| JDK                             | 21                                                                        |
| Gradle                          | 8.14.2                                                                    |
| Android Gradle Plugin (AGP)     | 8.7.0                                                                     |
| Kotlin                          | 2.0.21                                                                    |
| Google Services (Gradle plugin) | 4.5.0                                                                     |
| Android Emulator                | 37.1.11 (Netsim BLE virtual controller habilitado, requiere $\geq 36.5$ ) |
| Android Studio                  | 2025.3.1 (build AI-253.29346.138.2531.14876573)                           |
| Visual Studio Code              | 1.134.0                                                                   |

Esta combinación de versiones fue necesaria porque el toolchain original (Gradle 7.6.3 / AGP 7.3.0 / Kotlin 1.7.10) era incompatible con JDK 21 y con los mínimos que exige Flutter 3.44 (Gradle  $\geq 8.14.0$ , AGP  $\geq 8.6.0$ ).

## 8.2 Emuladores / dispositivos

- **Wear OS:** emulador `sdk_gwear_x86_64`.
- **Android (teléfono):** emulador `sdk_gphone64_x86_64`.
- Ambos emuladores comparten el controlador Bluetooth virtual (Netsim) del Android Emulator, lo que permite BLE real entre dos instancias de emulador en la misma máquina – sin necesidad de hardware físico.
- **Navegador de pruebas para la PWA:** Brave 151.1.93.138 (Chromium), usado tanto en modo normal como headless (automatización con Playwright), siempre a viewport 1920×1080. El User-Agent usado fue el de escritorio por defecto de Brave/Chromium – `pwa_tv` no hace ningún tipo de detección de User-Agent en su código (`app.js`), por lo que el comportamiento es idéntico sin importar el UA; no fue necesario forzar un UA de Smart TV.

## 8.3 Firebase

Proyecto `ironexx-extraordinaria`, con apps registradas para Web (`pwa_tv`) y Android (`com.ironexx.mobile_app`). Firestore en modo de prueba (reglas abiertas por tiempo limitado – decisión documentada por alcance de esta entrega, ver `docs/extraordinaria/checklist_seguridad_pwa.md`). El archivo `google-services.json` y el `apiKey` web sí están en el repositorio a propósito (no son secretos de servidor – ver justificación en la sección de seguridad E5 del código), para que el profesor pueda clonar y ejecutar el proyecto sin configuración adicional.

## 8.4 Procedimiento de ejecución

1. Clonar el repositorio y hacer checkout de la rama `evaluacion-2-extraordinaria`.
2. Levantar un emulador Wear OS y un emulador de teléfono Android (ambos con Netsim BLE).

3. `flutter run` en `ironexx_dispo/wearable_app` (seleccionar el emulador Wear OS).
4. `flutter run` en `ironexx_dispo/mobile_app` (seleccionar el emulador de teléfono).
5. Servir `ironexx_dispo/pwa_tv` con cualquier servidor estático (ej. `python -m http.server 8080`) y abrirlo en un navegador a 1920x1080.
6. En el reloj, presionar “Iniciar”; en el teléfono, presionar “Buscar reloj”; verificar que las 3 métricas aparecen en tiempo real. **La primera vez** que el teléfono se conecta a un reloj recién instalado, Android muestra una notificación del sistema (“Pairing request”) que hay que aceptar (“Pair & connect”) – es el emparejamiento BLE normal del sistema operativo, no un error de la app; en conexiones posteriores no vuelve a aparecer.
7. En el teléfono, seleccionar una máquina (búsqueda o chips de catálogo); verificar que la TV se actualiza sin recargar.

## 9 Inventario de evidencias

| ID          | Criterio | Descripción                                              | Qué demuestra                                   |
|-------------|----------|----------------------------------------------------------|-------------------------------------------------|
| E1-EV01     | E1       | Reloj transmitiendo las 3 métricas simultáneamente       | GATT Server + NOTIFY funcionando                |
| E1-EV02     | E1       | <code>dumpsys bluetooth_manager</code> del wearable      | Advertising activo, GATT server registrado      |
| E1-EV03     | E1       | Banner de alerta por umbral crítico                      | Lógica de umbrales por métrica                  |
| E1-EV04     | E1       | Transmisión simultánea de las 3 características          | Consistencia de datos en tiempo real            |
| E1-EV05     | E1       | <code>dumpsys bluetooth_manager</code> , conexión activa | GATT connection establecida                     |
| E2-EV00     | E2       | <code>dumpsys bluetooth_manager</code> del teléfono      | Cliente GATT conectado                          |
| E2-EV01     | E2       | Estado desconectado (antes de escanear)                  | Estado inicial correcto de la UI                |
| E2-EV02     | E2       | Teléfono conectado, recibiendo datos en vivo             | Decodificación correcta, valores en tiempo real |
| E3-EV01     | E3       | Grid 2x2 completo, 1920x1080, sin recortes               | Layout, safe zone, legibilidad mínima           |
| E3-EV02..04 | E3       | Navegación D-pad (foco inicial, derecha, abajo)          | Navegación por teclado/D-pad funcional          |
| E3-EV05     | E3       | Selección con Enter                                      | Interacción OK/Enter                            |
| E3-EV06     | E3       | Modo offline (Service Worker)                            | Cache First / funcionamiento sin red            |
| E4-EV01..03 | E4       | 3 corridas de sincronización teléfono → TV               | <code>onSnapshot()</code> en tiempo real, <2s   |
| E5-EV01     | E5       | Desconexión BLE a media conexión, sin crash              | Manejo robusto de fallos                        |

Todas las evidencias se encuentran en la carpeta **evidencias/** (fuera del repositorio, organizada por criterio: **E1\_wear**, **E2\_movil**, **E3\_pwa**, **E4\_firestore**, **E5\_seguridad**, **lighthouse**), y una copia de las imágenes usadas en este documento vive en **reporte\_extraordinaria/images/**.

## 10 Lighthouse

### 10.1 Resultados

Auditoría ejecutada con `npx lighthouse -preset=desktop` sobre **pwa\_tv** servido en **localhost:8080**.

|         | Performance | Accessibility | Best Practices | SEO        |
|---------|-------------|---------------|----------------|------------|
| Antes   | 99          | 100           | 100            | 91         |
| Después | 96          | 100           | 100            | <b>100</b> |

## 10.2 Hallazgo corregido

Lighthouse SEO señaló la ausencia de una meta descripción (`meta name="description"`), requisito básico de SEO para que buscadores/asistentes puedan indexar correctamente la página. Se agregó a `index.html`:

```
<meta name="description" content="Ironexx: pantalla Smart TV con catalogo de
maquinaria de construccion, sincronizada en tiempo real desde la app movil.">
```

Tras el cambio, SEO subió de 91 a 100. La variación de Performance (99  $\rightarrow$  96) está dentro del ruido normal de mediciones consecutivas de Lighthouse (no atribuible a este cambio, que es puramente de metadatos) – ambas corridas quedan en el rango “verde” ( $\geq 90$ ).

## 10.3 Evidencia

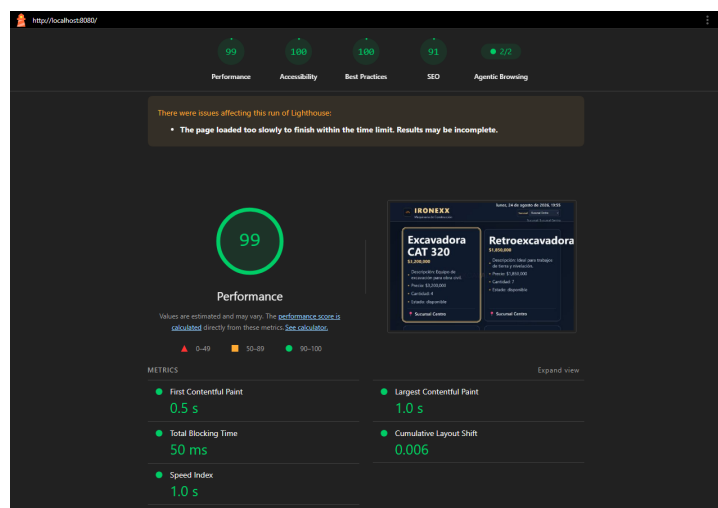


Figura 9: Reporte Lighthouse antes (SEO 91).

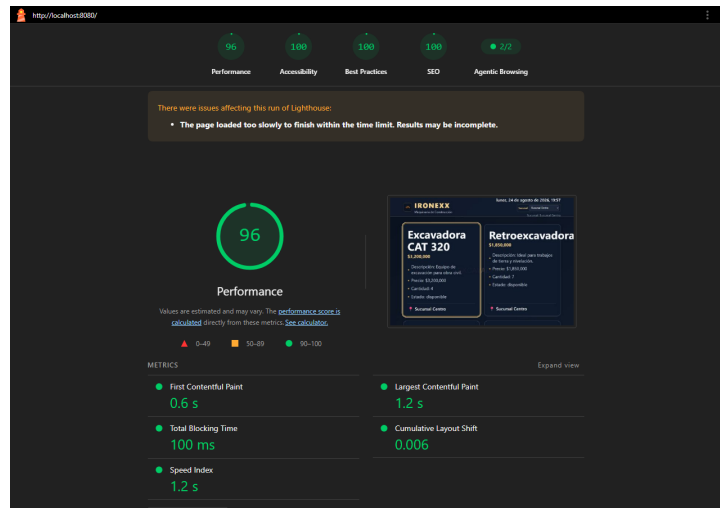


Figura 10: Reporte Lighthouse después (SEO 100).

## 11 Problemas encontrados y soluciones

| Problema                                                                                                            | Causa                                                                                                                          | Solución                                                                                                                                                                         | Resultado                                   |
|---------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------|
| Los emuladores no lo-graban conectarse por BLE entre sí (único punto señalado por el revisor).                      | Faltaba implementar el GATT Server real en el reloj (advertising + NOTIFY) y el filtrado por Service UUID en el teléfono.      | Se implementó <code>ble_peripheral</code> en el reloj y <code>flutter_blue_plus</code> en el teléfono, con UUIDs custom centralizados y advertising sin <code>localName</code> . | Conexión BLE end-to-end verificada en vivo. |
| <code>assembleRelease</code> fallaba en ambas apps Flutter.                                                         | Toolchain desactualizado (Gradle 7.6.3 / AGP 7.3.0 / Kotlin 1.7.10) incompatible con JDK 21 y con los mínimos de Flutter 3.44. | Se actualizó a Gradle 8.14.2, AGP 8.7.0, Kotlin 2.0.21.                                                                                                                          | Build reproducible en ambos proyectos.      |
| Error de compilación en el plugin <code>shared_preferences_android</code> (PluginRegistry.Registrar no encontrado). | Versión antigua del plugin, basada en el v1 embedding ya removido.                                                             | <code>flutter pub upgrade</code> (2.2.3 → 2.5.5).                                                                                                                                | Compilación limpia.                         |

| Problema                                                                              | Causa                                                                                                                                                                                                                             | Solución                                                                                                                                                     | Resultado                                                                                                                                          |
|---------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| La pantalla antigua del wearable fallaba con <code>SocketException</code> al iniciar. | Faltaba el permiso <code>INTERNET</code> en el manifest, y esa pantalla dependía de un login contra un backend que no aplica al alcance de esta evaluación.                                                                       | Se agregó el permiso y, tras confirmarlo con la usuaria, se eliminó por completo la pantalla de login y se dejó el reloj entrando directo al monitor BLE.    | Flujo del reloj simplificado y sin errores.                                                                                                        |
| UI del reloj desbordaba la pantalla / botones desproporcionados.                      | Tamaños fijos en píxeles no consideraban el <code>shortestSide</code> real del emulador Wear OS.                                                                                                                                  | Layout iterado con <code>MediaQuery.shortestSide * 0.56</code> y paddings reducidos, verificado visualmente en cada iteración.                               | UI centrada y proporcionada.                                                                                                                       |
| Al seleccionar una máquina en el teléfono no se reflejaba nada en la TV.              | La sincronización estaba conectada a la búsqueda por registro QR local, cuya lista empieza vacía (sin máquinas registradas).                                                                                                      | Se agregó un catálogo de ejemplo ( <code>_catalogoDemo</code> ) con chips de selección rápida que sincronizan por nombre aunque no exista un registro local. | E4 demostrable sin depender de un flujo QR previo.                                                                                                 |
| Grid de la PWA recortaba contenido (última fila y etiqueta de sucursal invisibles).   | Suma de alturas (título mínimo obligatorio + línea de precio duplicada + paddings) mayor al alto disponible del grid.                                                                                                             | Se eliminó la línea de precio duplicada y se redujeron paddings/márgenes internos, sin bajar ningún tamaño de fuente del mínimo exigido.                     | Grid completo y legible, verificado con capturas headless.                                                                                         |
| Botón “Buscar reloj” parecía no responder durante varias pruebas.                     | No era un bug de código: un diálogo de permiso de ubicación de Android aparecía tras cada intento y las capturas automáticas no lo alcanzaban a mostrar a tiempo; los toques repetidos caían sobre el diálogo, no sobre el botón. | Se identificó el diálogo revisando una captura intermedia y se aceptó explícitamente (“While using the app”).                                                | Documentado como lección de testing, no como defecto funcional – útil para no confundirlo con un bug real durante la sesión de evaluación en vivo. |

| Problema                                                                                                                                                                     | Causa                                                                                                                                                                                  | Solución                                                                                                                                                                                                                                                                                                                                                        | Resultado                                                                                                                |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Falsos negativos al medir el tiempo de sincronización E4 en las primeras corridas de prueba.                                                                                 | El script de prueba buscaba el elemento <code>.media-fallback-text</code> (que solo aparece si una imagen falla al cargar); el catálogo de ejemplo sí tiene imágenes de fondo válidas. | Se corrigió el script para verificar el estilo <code>backgroundImage</code> del fondo en su lugar.                                                                                                                                                                                                                                                              | Mediciones reales obtenidas (247 ms, 565 ms, 182 ms) – no era un bug del proyecto, sino de la instrumentación de prueba. |
| Credenciales reales (contraseña de correo personal, contraseña de base de datos, clave secreta de reCAPTCHA) encontradas hardcodeadas en el código y en el historial de git. | Código/commits antiguos nunca depurados antes de que el repositorio se hiciera público.                                                                                                | Ver detalle completo en la documentación de seguridad ( <a href="#">docs/extraordinaria/</a> ); se corrigieron las 4 ubicaciones en código para usar variables de entorno, y se reescribió el historial de git ( <code>git filter-repo</code> , dos pasadas) tras consentimiento explícito de la dueña de las credenciales, sin perder ningún commit funcional. | Historial limpio, repositorio público verificado sin secretos, funcionalidad intacta.                                    |
| Al corregir el recorte del grid (fila anterior de esta tabla) se había reducido por error la zona segura vertical de 54px a 38px para ganar espacio.                         | La guía especifica un valor exacto (“Safe zone: 96px horizontal y 54px vertical”), y esa corrección inicial no lo respetó.                                                             | En una auditoría final línea por línea contra la guía se detectó la desviación; se revirtió a 54px exactos y se ajustaron en su lugar solo <code>padding/gaps</code> internos de tarjeta hasta que las 4 tarjetas volvieran a caber completas.                                                                                                                  | Zona segura conforme a la guía (96px/54px) y grid sin recortes, verificado de nuevo con capturas.                        |

| Problema                                                                                                                                                                                                                                                                                        | Causa                                                                                                                                                                                                                                                                                                                                                                                | Solución                                                                                                                                                                                                                                                                                                                                                                                                              | Resultado                                                                                                                                                   |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Tres carpetas del repositorio ( <code>backend</code> , <code>maquinaria-construccion</code> , <code>PRODUCCION_LISTO/frontend</code> , del sitio web de venta de maquinaria, fuera del alcance de esta evaluación) estaban registradas como enlaces de submódulo sin <code>.gitmodules</code> . | Eran proyectos locales sueltos agregados sin querer como submódulo; en un <code>git clone</code> nuevo esas carpetas aparecerían vacías.                                                                                                                                                                                                                                             | Se incorporaron <code>maquinaria-construccion</code> y <code>PRODUCCION_LISTO/frontend</code> como contenido normal del repositorio en una rama aparte ( <code>codigo-completo-sitio-web</code> ); a <code>backend</code> se le detectó además un <code>.env</code> con la contraseña de base de datos ya identificada como comprometida, así que en vez de subirlo se entregó por separado (copia local, no GitHub). | No afecta a <code>ironexx_dispo</code> (árbol normal, sin submódulos, 100 % reproducible desde un solo clone) ni a la rama evaluada de esta extraordinaria. |
| La conexión BLE, que ya había funcionado en vivo, dejó de conectar de forma consistente en una sesión posterior (“Timed out after 15s”), incluso reiniciando la PC por completo y en un proyecto de referencia distinto.                                                                        | El emparejamiento (bonding) BLE a nivel de sistema operativo, disparando automáticamente por Android al suscribirse a NOTIFY por primera vez, tarda cerca de 20s en completarse; el timeout de <code>device.connect()</code> del lado del teléfono estaba fijado en 15s, así que la app cancelaba la conexión un instante antes de que el emparejamiento terminara, en cada intento. | Se descartó metodicamente RAM, direcciones fijas de <code>netdim.ini</code> , y una versión corrupta del Android Emulator (se actualizó 36.5.11 → 37.1.11, mismo síntoma) antes de aislar la causa real con captura de <code>logcat</code> en ambos dispositivos durante el intento de conexión. Se subió el timeout de <code>device.connect()</code> de 15s a 30s en <code>ble_service.dart</code> .                 | Conexión, emparejamiento y recepción de datos en vivo verificados de nuevo end-to-end.                                                                      |

## 12 Guión de demostración

Preparación previa a la sesión: tener abiertos el repositorio (rama `evaluacion-2-extraordinaria`), el emulador Wear OS, el emulador de teléfono, la PWA de la TV en el navegador (1920x1080) y la consola de Firebase/Firestore.

1. **E1 – Wear OS:** mostrar el reloj en estado “Detenido”; presionar “Iniciar” y mostrar las 3 métricas actualizándose cada segundo. Explicar el GATT Server, el Service UUID y por qué no se anuncia `localName`.

2. **E2 – Teléfono:** presionar “Buscar reloj”; mostrar el escaneo filtrado por Service UUID, la conexión, y las 3 métricas llegando en tiempo real e idénticas a las del reloj.
3. **Prueba de desconexión (E5):** presionar “Detener” en el reloj a media conexión; mostrar que el teléfono no truena y cambia a “Sin datos recientes”.
4. **E3 – PWA Smart TV:** mostrar el grid 2x2 completo; navegar con las flechas del teclado (D-pad) mostrando el foco moviéndose y sin romperse en los bordes; presionar Enter sobre una tarjeta.
5. **E4 – Firestore:** desde el teléfono, seleccionar una máquina distinta (búsqueda o chip de catálogo); mostrar que la TV se actualiza sola, sin recargar, en menos de 2 segundos.
6. **Offline (E3/E5):** en el navegador de la TV, activar modo sin conexión (DevTools) y recargar; mostrar que la app sigue funcionando desde el Service Worker.
7. **Seguridad (E5):** mostrar la documentación en docs/extraordinaria/ y, si se pide, explicar el hallazgo y la limpieza del historial de git (sin exponer credenciales reales durante la demo).
8. **Lighthouse:** mostrar el reporte antes/después y explicar el fix de SEO (meta description).
9. **Preguntas y modificación en vivo:** quedar disponible para que el profesor pida explicar o modificar cualquier parte del código mostrado.

## 13 Conclusiones

### 13.1 Estado final

Los 6 ejes de la evaluación (E1–E6) están implementados y verificados en vivo: el reloj publica 3 métricas relevantes al dominio de maquinaria pesada por BLE GATT/NOTIFY; el teléfono se conecta, decodifica y muestra esos datos en tiempo real, y maneja la desconexión sin fallar; la PWA de la Smart TV cumple resolución, safe zone, navegación D-pad y modo offline; la sincronización vía Firestore entre teléfono y TV ocurre en menos de 2 segundos en las 3 corridas medidas; y se corrigió además un incidente de seguridad real (credenciales expuestas) fuera del alcance original de la rúbrica, por responsabilidad propia hacia terceros afectados.

### 13.2 Limitaciones reales

- **Importante, con fecha límite real:** Firestore corre en modo de prueba (reglas abiertas), que Firebase expira automáticamente 30 días después de creado el proyecto (creado el 24-ago-2026  $\Rightarrow$  expira alrededor del 23-sep-2026). Si la sesión de evaluación ocurre después de esa fecha, **Firestore dejaría de leer/escribir por completo y E4 fallaría en vivo** aunque el código sea correcto. Se recomienda revisar/actualizar las reglas en Firebase Console antes de la sesión (o antes de esa fecha) para evitar depender del modo de prueba.

- El registro de máquinas por código QR existe en el código pero no se probó exhaustivamente en el emulador (análisis de código confirmó su corrección lógica); la demostración de E4 se apoya en el catálogo de ejemplo para no depender de ese flujo.
- `tv_native_app` existe en el repositorio pero no se integró a este flujo – se priorizó `pwa_tv` como la implementación completa y evaluable.

### 13.3 Aspectos que pueden mejorarse

- Restringir las reglas de Firestore por autenticación antes de cualquier uso más allá de esta evaluación.
- Agregar un historial de estados en Firestore (colección aparte) si en el futuro se necesita trazabilidad de selecciones, con el mismo esquema de retención de 30 días ya usado en el resto del sistema.
- Probar el flujo de registro por QR de punta a punta en el emulador, no solo por revisión de código.